

Magic methods

(CC-BY: Maria Skeppstedt)

Magic methods

- ▶ These methods have special meanings for Python
- ▶ Name starts and ends with a double underscore.

Will only give a few examples here.

__init__

The `__init__` method is a very familiar magic method. It creates a new instance of the class

```
1 class TimePasser:
2
3     def __init__(self, day_hours = 24):
4         self.day_hours = day_hours
5         self.passed_hours = 0
6
7 earth = TimePasser(24)
8 planet_2 = TimePasser(24)
9 planet_3 = TimePasser(32)
```

The `__str__` method is, for instance, called when `print` is called.

```
1 class TimePasser:
2
3     def __init__(self, day_hours = 24):
4         self.day_hours = day_hours
5         self.passed_hours = 0
6
7     def get_days(self):
8         return int(self.passed_hours / self.day_hours)
9
10    def get_hours_today(self):
11        return self.passed_hours % self.day_hours
12
13    def __str__(self):
14        return \
15            f"TimePasser completed {self.get_days()} days and
16            {self.get_hours_today()} hours"
17
18 earth = TimePasser(24)
19 print(earth)
```

TimePasser completed 0 days and 0 hours

__eq__

Notice the difference between == and is:

```
1 class TimePasser:
2
3     def __init__(self, day_hours = 24):
4         self.day_hours = day_hours
5         self.passed_hours = 0
6
7     def __eq__(self, other):
8         return self.day_hours == other.day_hours and \
9             self.passed_hours == other.passed_hours
10
11 earth = TimePasser(24)
12 planet_2 = TimePasser(24)
13
14 print("Equals?", earth == planet_2)
15 print("Same object?", earth is planet_2)
```

Equals? True

Same object? False

__call__

Use the same syntax, as for a function:

```
1  class TimePasser:
2
3      def __init__(self, day_hours = 24):
4          self.day_hours = day_hours
5          self.passed_hours = 0
6
7      def __call__(self, hours_to_add=1):
8          self.passed_hours += hours_to_add
9          return self.passed_hours
10
11  earth = TimePasser(24)
12  planet_2 = TimePasser(24)
13  earth(3)
14  planet_2()
15  print(earth)
16  print(planet_2)
```

TimePasser completed 0 days and 3 hours

TimePasser completed 0 days and 1 hours

__repr__ and __lt__

```
1 class TimePasser:
2     def __init__(self, day_hours = 24, passed_hours = 0):
3         self.day_hours = day_hours
4         self.passed_hours = passed_hours
5
6     def __repr__(self):
7         return f"TimePasser(day_hours={self.day_hours},
8             passed_hours={self.passed_hours})"
9
10    def __lt__(self, other):
11        return self.passed_hours > other.passed_hours
12
13 earth = TimePasser(24)
14 planet_2 = TimePasser(24)
15 planet_3 = TimePasser(32)
16 earth(3)
17 planet_2()
18 print(sorted([planet_2, planet_3, earth]))
```

```
[TimePasser(day_hours=24, passed_hours=3),
TimePasser(day_hours=24, passed_hours=1),
TimePasser(day_hours=32, passed_hours=0)]
```

The **iterator** protocol requires `__iter__` and `__next__`.

The **iterable** protocol requires `__iter__`.

(`__iter__` must return an iterator)

```
1 class TimePasser:
2     def __init__(self, day_hours = 24):
3         self.day_hours = day_hours
4         self.passed_hours = 0
5
6     def __iter__(self):
7         return self
8
9     def __next__(self):
10        self.passed_hours += 1
11        return self.passed_hours
12
13 earth = TimePasser(24)
14 planet_3 = TimePasser(32)
15 for hours in earth:
16     next(planet_3)
17     print("*****", "Earth hour:", hours, "*****")
18     print("Earth:", earth)
19     print("Planet_3:", planet_3)
```

```
1 class TimePasser:
2     def __init__(self, day_hours = 24):
3         self.day_hours = day_hours
4         self.passed_hours = 0
5
6     def __iter__(self):
7         return self
8
9     def __next__(self):
10        self.passed_hours += 1
11        return self.passed_hours
12
13 earth = TimePasser(24)
14 planet_3 = TimePasser(32)
15 for hours in earth:
16     next(planet_3)
17     print("*****", "Earth hour:", hours, "*****")
18     print("Earth:", earth, "\nPlanet_3:", planet_3)
```

```
***** Earth hour: 1 *****
Earth: TimePasser completed 0 days and 1 hours
Planet_3: TimePasser completed 0 days and 1 hours
...
***** Earth hour: 178235 *****
Earth: TimePasser completed 7426 days and 11 hours
Planet_3: TimePasser completed 5569 days and 27 hours
```

Note that Python doesn't care if TimePasser inherits from a superclass.

As long as we follow the protocol, it will work

```
1 class TimePasser:
2     def __init__(self, day_hours = 24):
3         self.day_hours = day_hours
4         self.passed_hours = 0
5
6     def __iter__(self):
7         return self
8
9     def __next__(self):
10        self.passed_hours += 1
11        return self.passed_hours
12
13 earth = TimePasser(24)
14 planet_3 = TimePasser(32)
15 for hours in earth:
16     next(planet_3)
17     print("*****", "Earth hour:", hours, "*****")
18     print("Earth:", earth)
19     print("Planet_3:", planet_3)
```

But we can subclass the abstract class `Iterator`, if we want to:
Then, the superclass will implement a default `__iter__` for us.

```
1 from collections.abc import Iterator
2
3 class TimePasser(Iterator):
4     def __init__(self, day_hours = 24):
5         self.day_hours = day_hours
6         self.passed_hours = 0
7
8     def __next__(self):
9         self.passed_hours += 1
10        return self.passed_hours
11
12 earth = TimePasser(24)
13 planet_3 = TimePasser(32)
14 for hours in earth:
15     next(planet_3)
16     print("*****", "Earth hour:", hours, "*****")
17     print("Earth:", earth)
18     print("Planet_3:", planet_3)
```
